Zeldiablo

Date de création	@28 mai 2025 09:11
Étiquettes	

https://github.com/MartinGouthier/2025_Zeldiablo_MartinGouthier

Untitled design

https://www.canva.com/design/DAGpXqD6ZVg/JMUZY2ZXbQD063P8EtMBMQ/edit?utm_content=DAGpXqD6ZVg&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton



Oral

Plan de passage :

- Jassem : Présentation de la base du projet avec la 1 ère itération
- Logan : Présentation de la deuxième itération (case a effet + pièges)
- Alexis : Présentation de la troisième itération (déplacement monstre)

Première itération :

Fonctionnalité 6.1 : mort du héros

Conception : Jassem, Martin

Implémentation : Yanis, Alexis, Logan

Séquence : Logan, Alexis, Jassem

Classe : Logan, Alexis

Tests : Alexis, Martin

Classe Perso——————————

Attributs à créer :

- vie (par défaut 5) dans Perso

Méthodes à créer :

- `Perso(int dx, int dy, int vie)`
- `public int getVie()`
- `public void modifierVie(int vie)`
- `public boolean estVivant()`

Méthodes à modifier :

- `Perso(int dx, int dy)` : ajouter un ligne vie = default (5)

- `etreFini()` : modifier les conditions d'arrêt du jeu (si estVivant == false)

Tests à réaliser :

- Vérifier que `modifierVie`, modifie la vie
- Vérifier que `estVivant`, affiche vivant quand il est vivant et mort quand il est mort
- Vérifier que le constructeur de Perso, le met aux bonnes coordonnées et aux bon PV

classe LabyDessin ————————

`dessinerJeu()` : créer 2 scénarios, si LabyJeu.etreFini() on dessine l'écran de fin, sinon pn dessine le jeu

classe LabyJeu————————-

`EtreFini` : modifier la méthode pour qu'il teste **`getPerso.estMort()`**

Diagramme de classes :

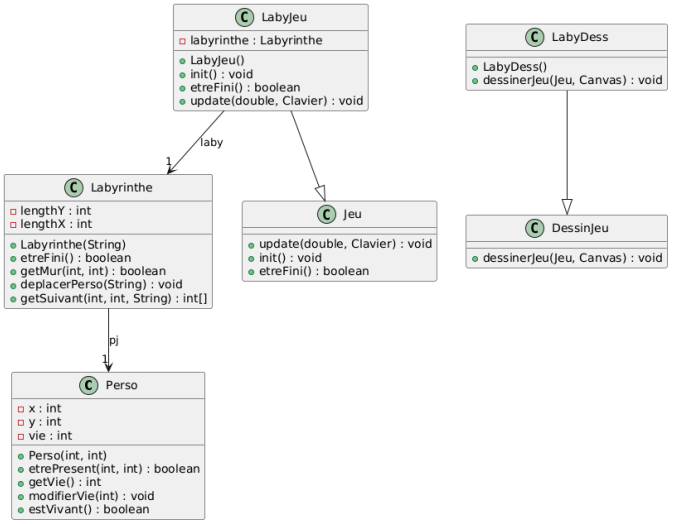
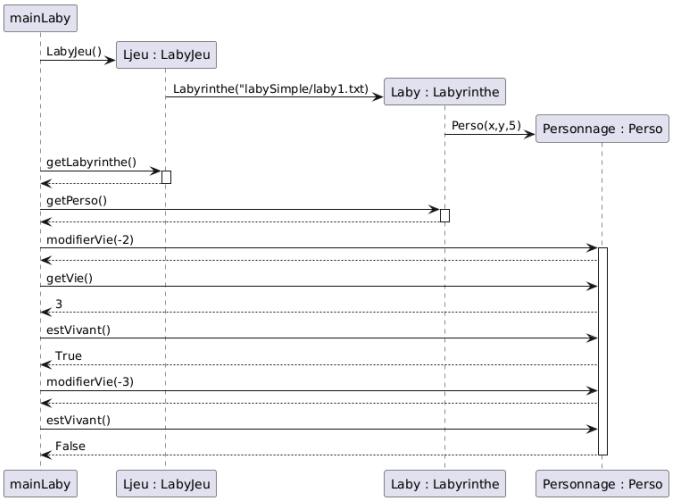
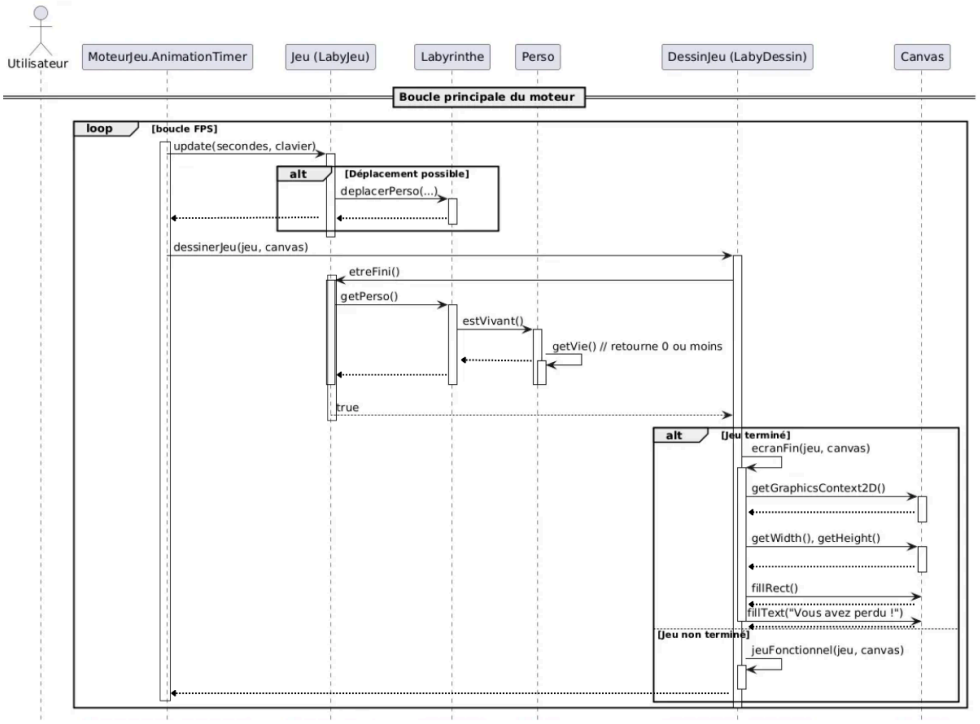


Diagramme de séquence :



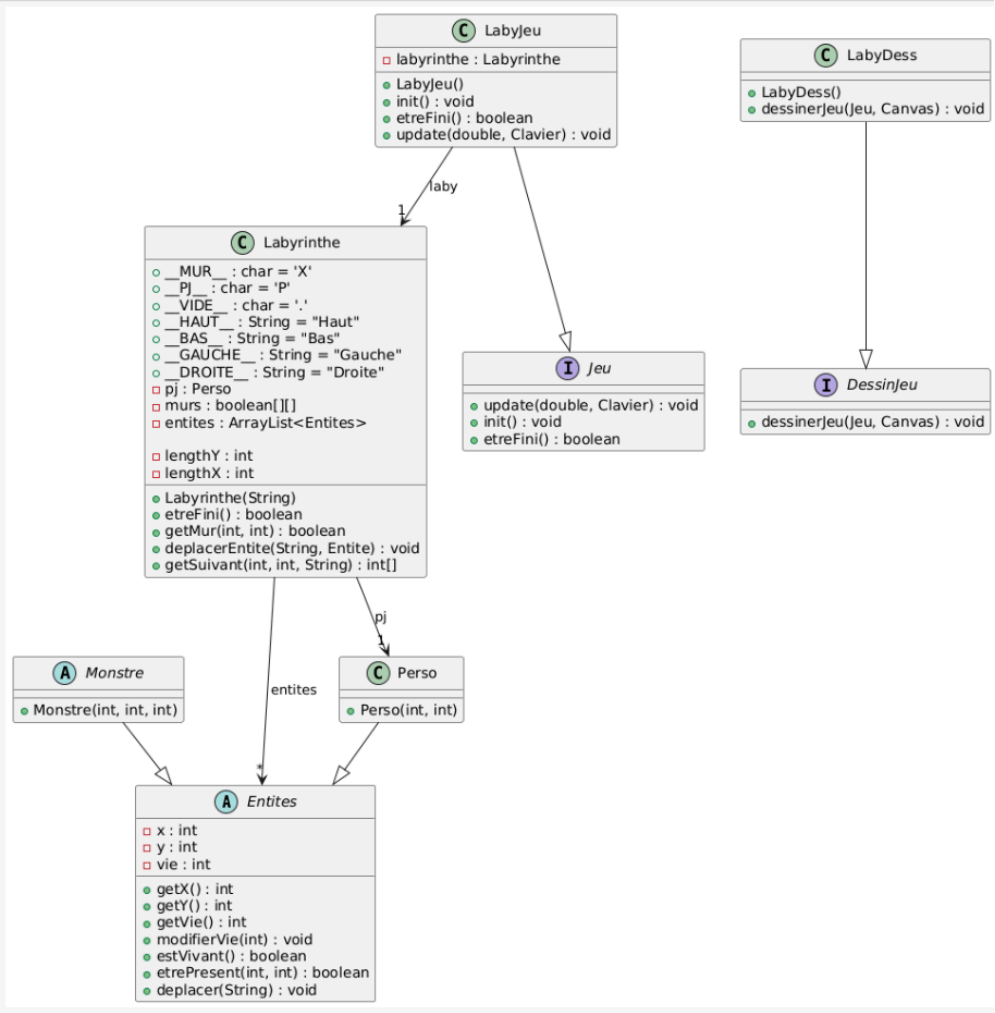
Scénario de mort du personnage:



Nous avons choisis d'implémenter cette fonctionnalité afin de s'orienter vers un moyen de terminer le jeu (de perdre). Nous avons pour objectif de concrétiser les monstres et toute autre entités dans notre jeu.

Fonctionnalité 2 : Monstre

diagramme de classe :



Nous avons choisis d'implémenter les monstres dans notre jeu, pour l'instant, la classe monstre est abstraite dans l'objectif d'accueillir plusieurs monstres. Dans le cas contraire, il suffira de rendre cette classe non abstraite. Les monstres et le personnage extends tous les deux la classe Entites. Ils possèdent 3 attributs, une position (décomposé avec la position x et la position y) ainsi que des points de vie qu'ils pourront tous les deux perdre.

Classes à créer :

- Classe abstraite "Entité"
- Classe "Monstre" extends "Entité" (représenter par un cercle violet)

Modifications engendrés par les monstres :

- Modification classe "Perso" extends "Entité"
- Déplacement des méthodes de Perso dans Entité
- Ajout d'un attribut final dans Labyrinthe pour les monstres ("M")
- Modification des fichiers "labyrinthe" pour ajouter un monstre à chaque fois
- Modification de la méthode `deplacerPerso()` de la classe "Perso" pour considérer les monstres

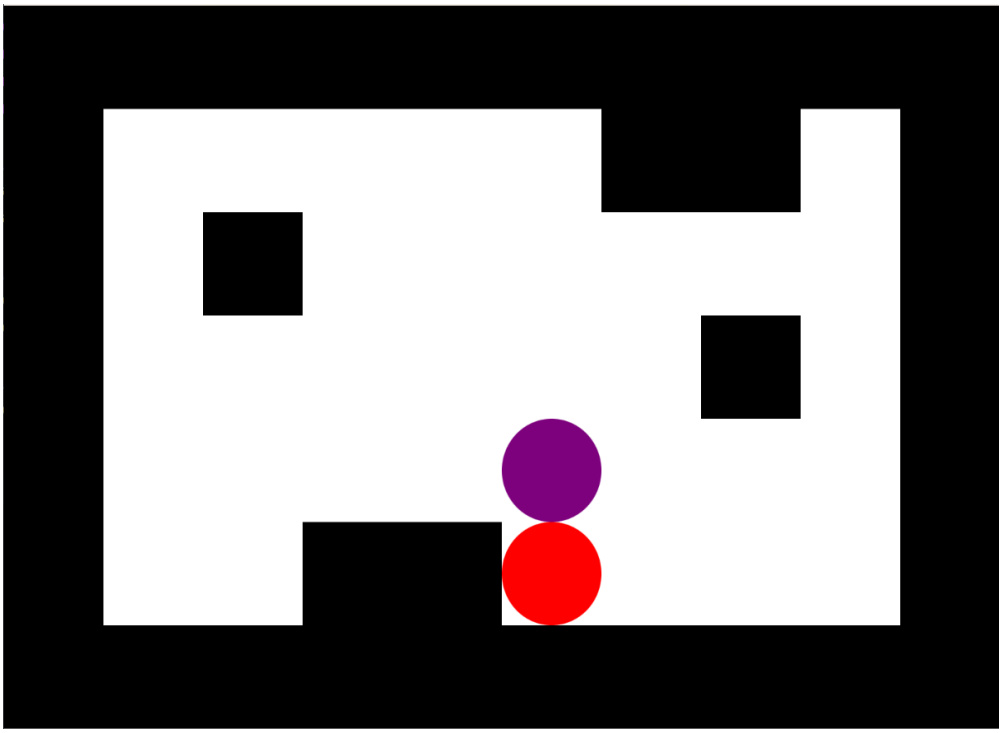
Tests à réaliser :

`le monstre se trouve sur la case indiquée dans le fichier labyrinthe.`

`le monstre ne se trouve pas sur la même case que le personnage.`

`le monstre constitue un obstacle pour le personnage.`

Screens :



Cette itération à donc servi à ajouter des points de vie ainsi que les monstres. Nous avons pour objectif futur d'implémenter les cases a effet (comme les pièges) et des monstres qui se déplacent vers le joueur afin d'orienter le jeu comme un jeu de survie, où l'ont doit se déplacer correctement.

Deuxième itération :

Conception : Martin, Yanis, Alexis

Implémentation : Jassem, Martin, Logan

Séquence : Alexis

Classe : Yanis

Tests : Alexis

Fonctionnalité 1.4 : Case à effets

Deux options s'offrent à nous :

- 1 classe `CaseAEffet` prenant un argument `TypeAEffet`
- 1 classe `CaseAEffet` et des classes `CaseXX extends CaseAEffet`

Nous avons choisi :

- 1 classe `CaseEffet` et des classes `CaseXX extends CaseAEffet`

Classes à créer :

- Classe `CaseEffets`
- Classe `CasePiege`

Classe Labyrinthe——————————

Attributs à créer :

- liste de cases à effet

Méthodes à créer :

- Génerer des `CaseAEffet` dans le constructeur `Labyrinthe()`

Méthodes à modifier :

- Gérer collision avec case dans `deplacerEntite()`

Classe CaseEffet——————————

Attributs à créer :

- x
- y

Méthodes à créer :

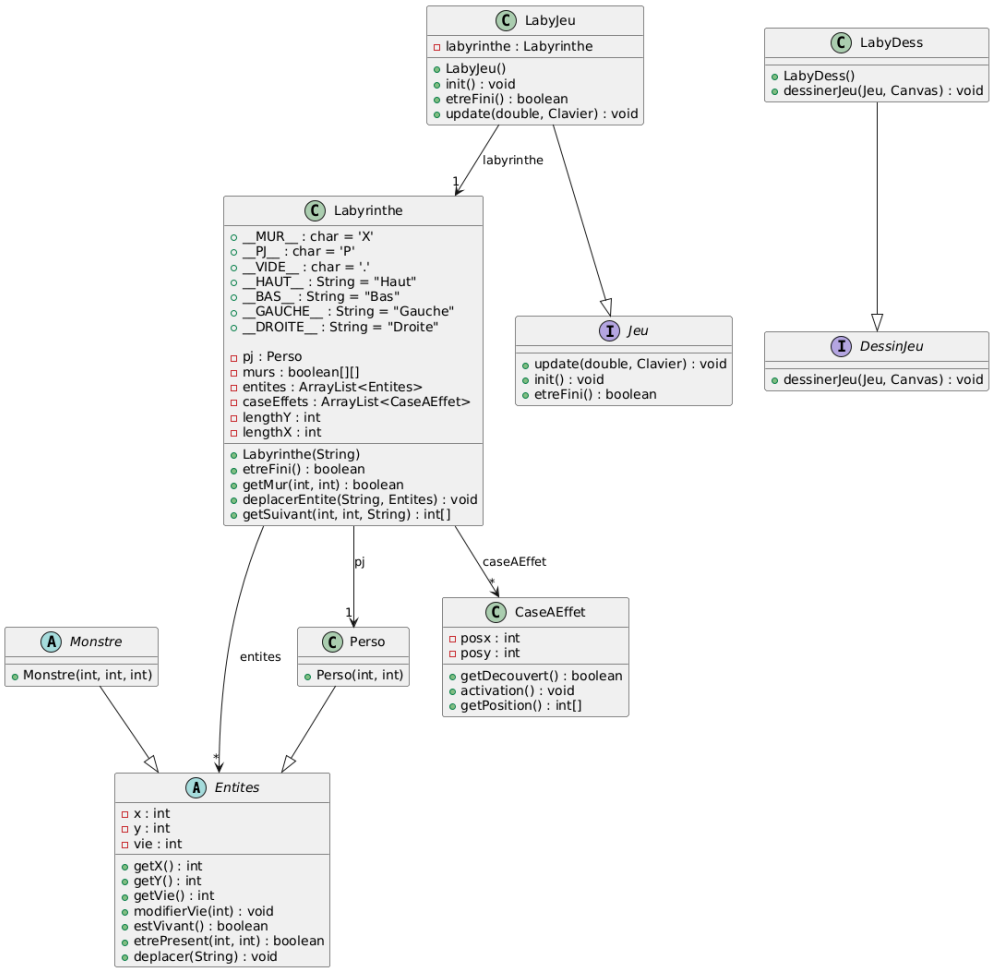
- `activation(Entite)`
- `getPosition()` (retourne un tableau de coordonnée [x,y])
- `getDecouvert()` (retourne le boolean decouvert)

Critères de validation :
• Les cases avec effet sont des cases traversables.
• Lorsqu'un personnage (héros ou monstre) marche sur cette case, elle déclenche immédiatement un effet
• cet effet se traduit pour le moment, par un affichage dans la console à préciser.

Tests

`les cases sont traversable par le joueur.`

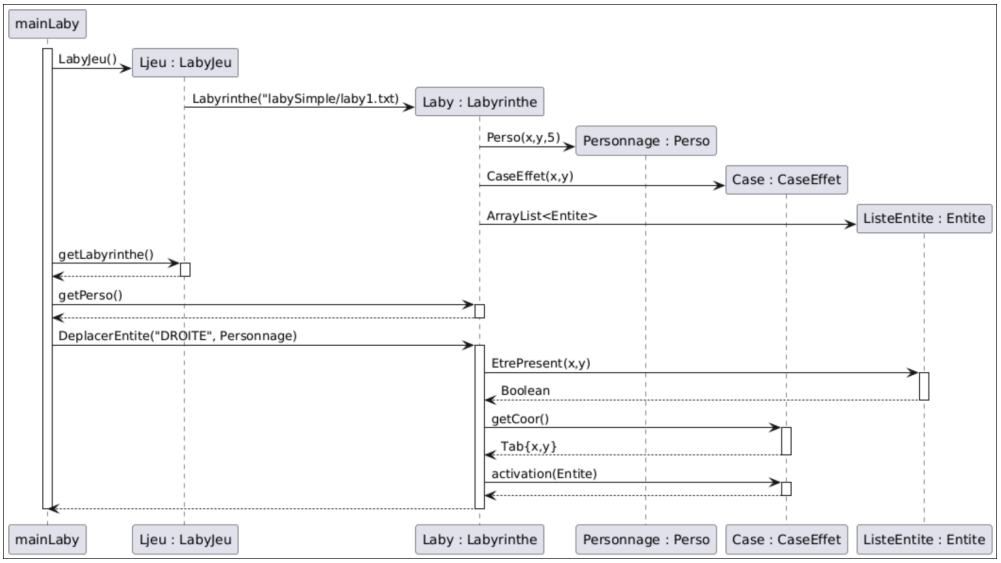
diagramme de classe :



Une case a effet (représenter par la classe CaseAEffet) est caractérisé par une position x ainsi qu'une position y. De plus, les différentes cases pourront posséder un attribut "decouvert", un booléen qui nous informera si le joueur à déjà marché sur la case ou non (d'ou la méthode getDecouvert() qui renverra toujours "true" dans le cas des CaseAEffet). Chaque case possède une méthode activation qui aura un effet différent selon la case (ici simplement un message dans la console).

CodeDuDiagramme

diagramme de séquence de activation d'une case à effet :



Après l'initialisation du jeu et des éléments de celui-ci, la méthode DeplacerEntite agit dans Labyrinthe et celle-ci vérifie d'abord s'il y a la présence d'un Entité dans la Arrayliste Entite qui ce trouve dans Labyrinthe pour ça elle utilise la méthode EtrePresent qui est dans la classe Entite.

Maintenant certain qu'il n'y a pas entité vers la prochaine position on peut vérifié s'il y a une case à effet avec la méthode getCoor( ) qui agit sur une liste de case dans Labyrinthe. Si les coordonnées reçus sont identique à celle de la prochaine position alors on active l'effet de la case.

Fonctionnalité 1.5 : Cases piégées

Classes à créer :

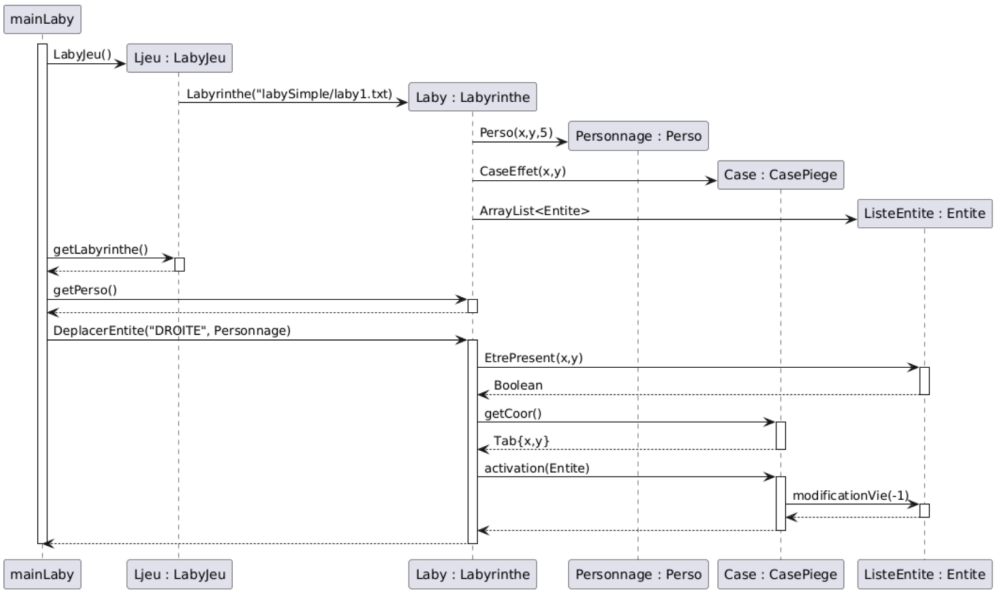
- `CasePiege exends CaseEffet`

Classe CasePiege——————————

Méthodes à créer :

- `activation(Entite)`

diagramme de séquence de activation d'une case piège :

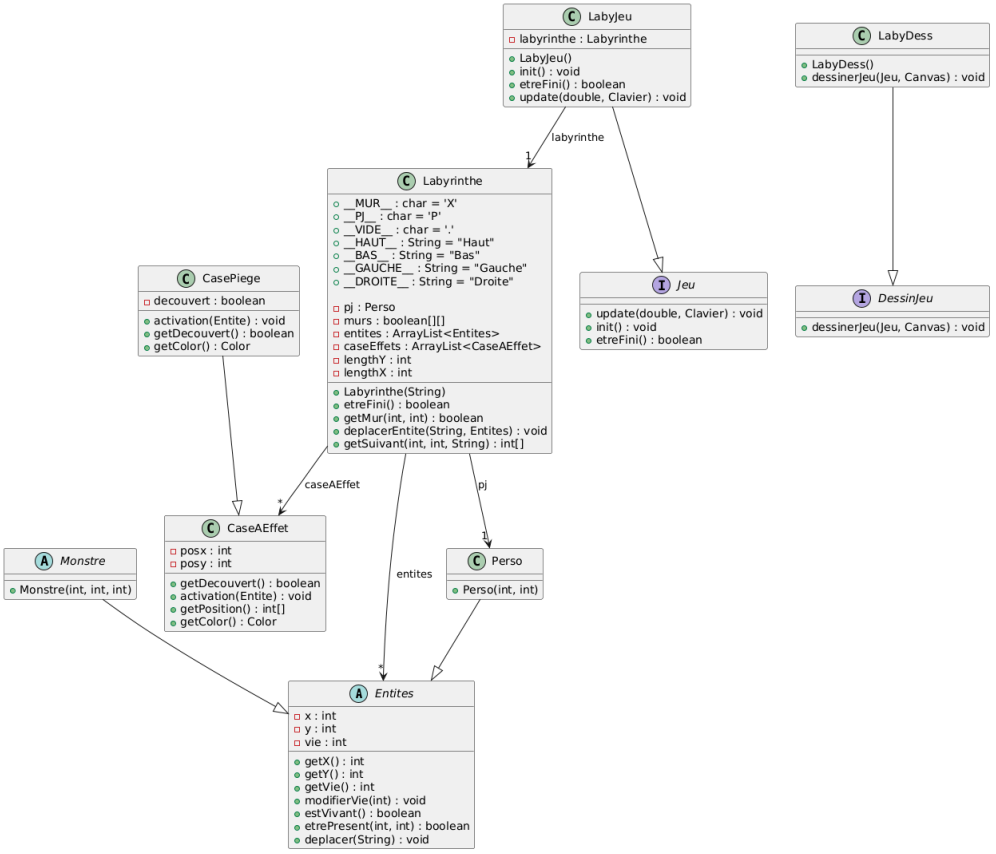


Ici le diagramme n'est pas si différent car case piège hérite de Case à effet donc du fait que l'Arraylist utilisé dans Labyrinthe pour les pièges elle peut prendre les case pièges.

l'activation des case à effet n'affiche qu'un message dans le log alors que celui des case piège fait pareil mais en ajoutant un appel à modificationVie et enlève donc 1 PV au Personnage.

S'il le monstre ce prend un piège il perd un point de vie et si ses points de vie arrive à 0, il est enlevé de la liste c'est à dire qu'il disparait du jeu.

diagramme de classes :



Au court de cette itération, nous avons ajouté la classe CaseAEffet ainsi que la classe CasePiege. La classe CasePiege extends la classe CaseAEffet afin de factoriser les cases. De cette manière, il est plus facile de gérer toutes les cases à effet si nous voulons en ajouter de nouvelles. Une case piège possède une position x, une position y et enfin, un attribut "decouvert". Cet attribut est un booléen qui, comme dit précédement sert à savoir si le joueur à déjà marché sur cette case une fois. Ceci est très utile pour les cases pièges puisqu'elles changent de couleur quand le joueur les à découvert (elles deviennent visible).

Critères de validation :

• Les dégâts ne se déclenchent que lorsque le joueur arrive sur la case, il peut ensuite rester sur cette case aussi longtemps qu'il le souhaite sans subir d'autres dégâts.
• Lorsque la case devient à nouveau vide, le piège se réamorce.
• (optionnel) Lorsqu'une case a été détectée comme un piège, elle s'affiche différemment dans le labyrinthe (en brun) ce qui permet au joueur d'en tenir compte ensuite.

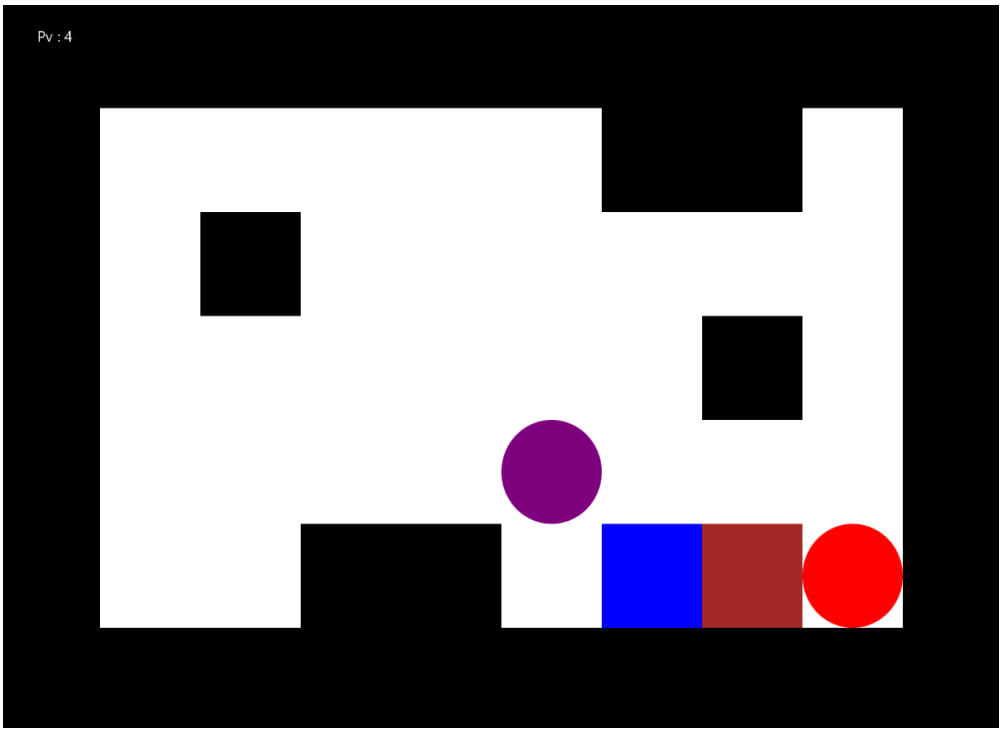
Tests à réaliser :

le joueur prend des dégats lorsqu'il est sur la case une seul fois.

la case piege ce réactive lors que le joueur quitte la case.

la case piege fait perdre un point de vie au joueur.

Screen



Troisième itération :

Conception : Logan, Alexis, Martin

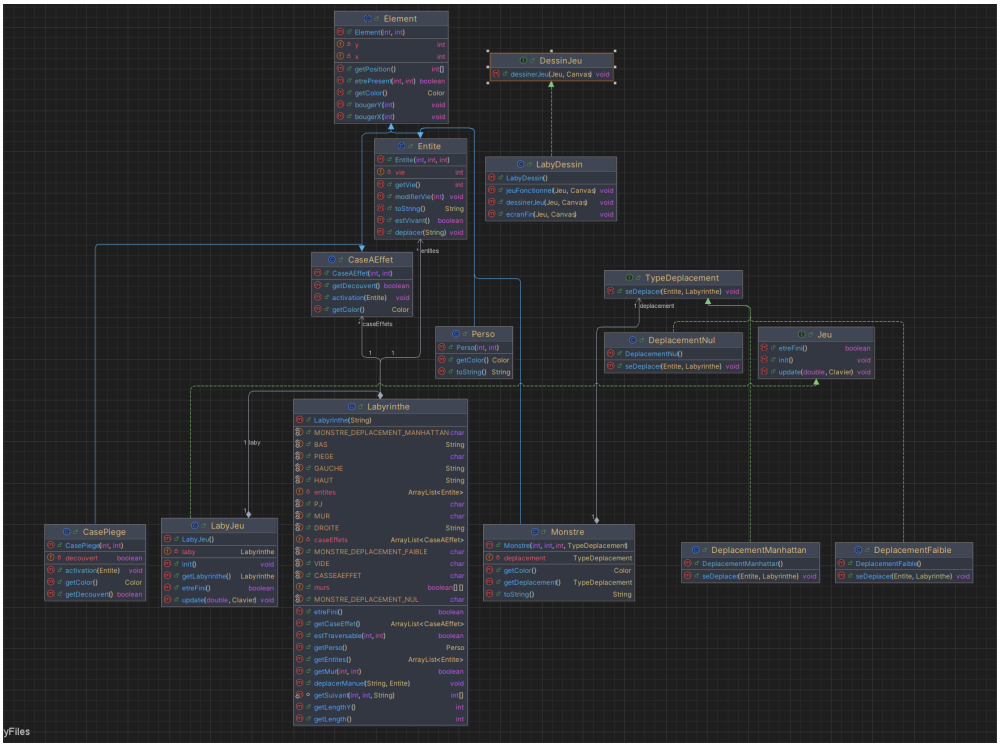
Implémentation : Yanis, Jassem

Séquence : Alexis

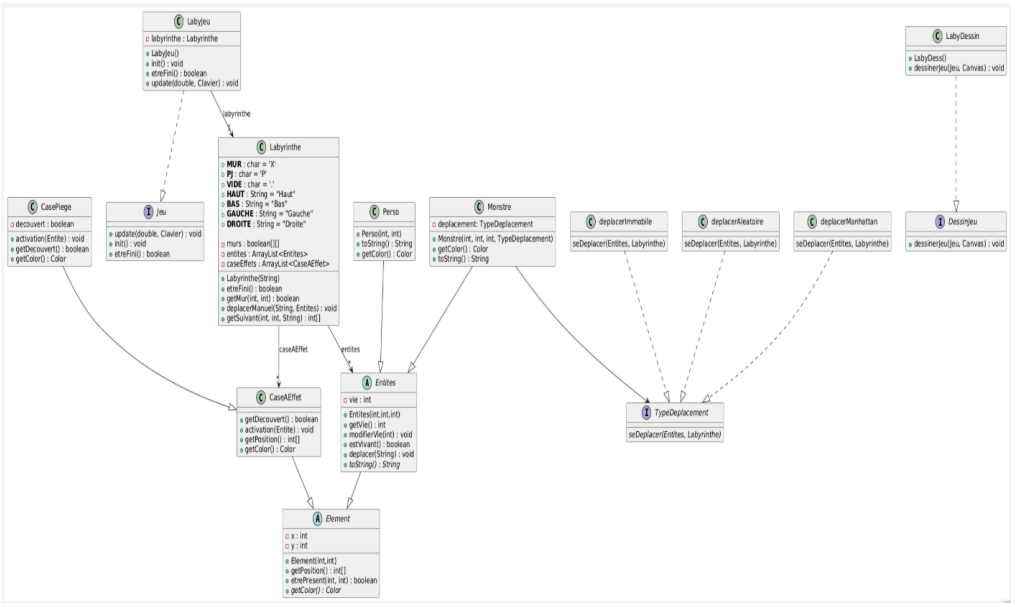
Classe : Martin

Tests : Logan, Alexis

diagramme de classe :



DiagrammeIntelliJ



DiagrammeClasse

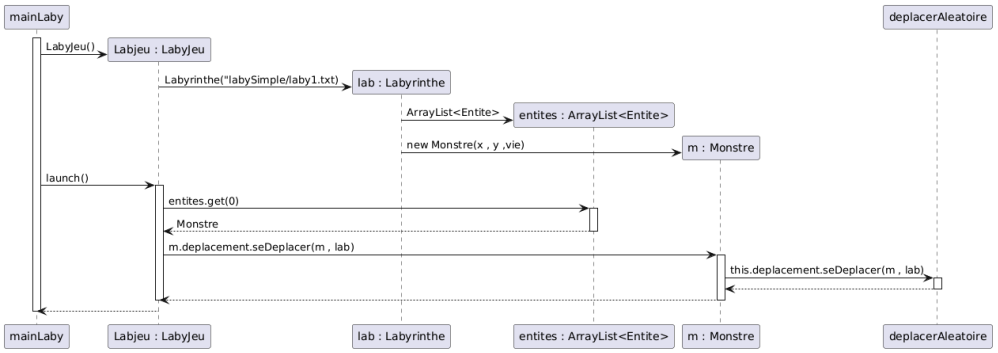
Code

Dans un premier temps, nous avons rassembler les entités ainsi que les cases à effets à l'aide de la classe Element car ces deux éléments sont similaires ils ont toutes les deux les attributs x et y et la méthode getColor. Puis, l'ajout d'un élément sera maintenant bien plus rapide et bien plus efficace et la gestion des éléments sera plus simple.

Ensuite, nous avons modifié la façon dont les coordonnées sont renvoyées dans quelques classes. Maintenant, c'est un tableau d'entier, auparavant, il y avait un getter pour la coordonnée x ainsi que pour la coordonnée y. Nous avons fait cela pour faciliter et uniformiser notre travail car certaine classes utilisaient une méthode et puis certaines utilisaient l'autre.

Enfin, pour implémenter les déplacement, nous avons créée une interface “TypeDeplacement” qui sera implémenter par tous les types de déplacements afin d’en ajouter autant que l’on veut. Il suffira de redéfinir la méthode “seDeplacer” de chaque classe qui implémente l’interface.

diagramme de séquence de seDeplacer :



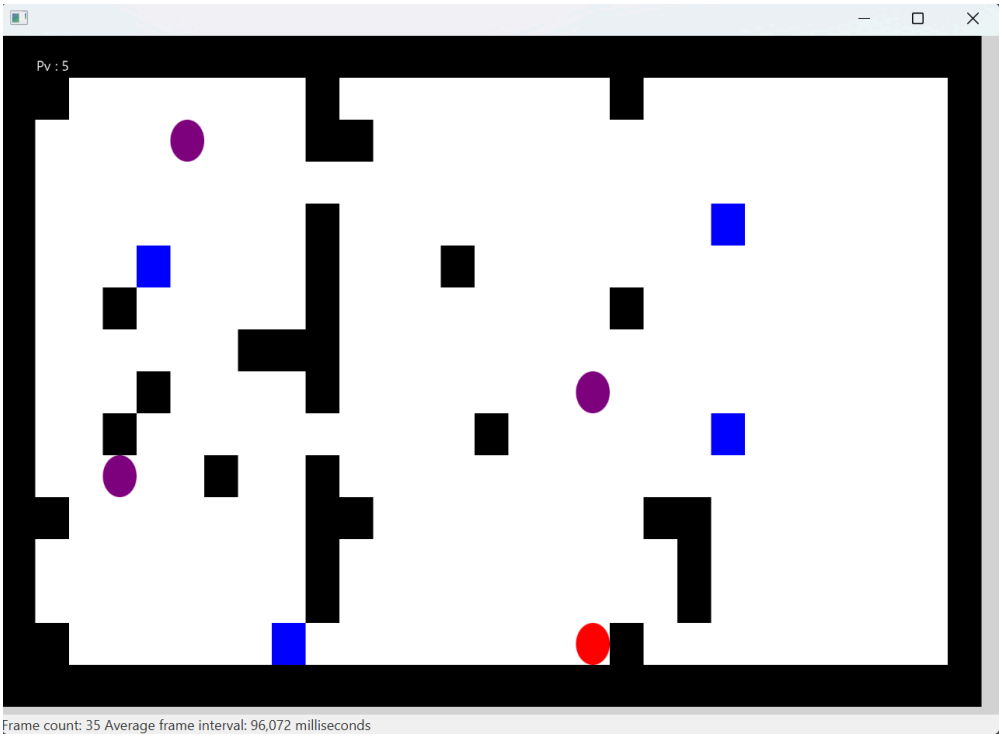
DiagrammeSequence

Code

Pour comprendre comment marche le déplacement d’un monstre, il faut ce rendre dans la boucle du jeu qui ce trouve dans LabyJeu dans la méthode update( ) qui vérifie et modifie les variables comme le joueur et les monstres pour chaque frames.

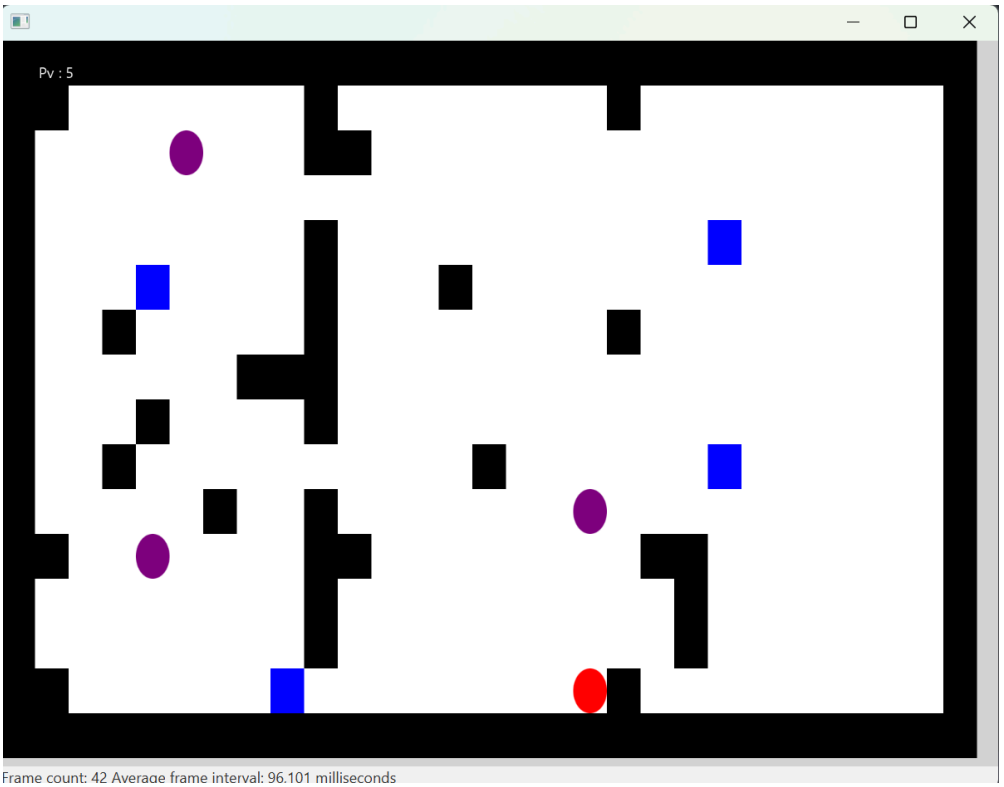
Pour cela, le main créer labyjeu pour ensuite le launch( ) qui lance la boucle de jeu dans MoteurJeu. Ainsi dans la boucle, il va récupérer chaque Entité de la liste, ici seul le monstre s’y trouve. Pour ensuite exécuter chaque Type de déplacement pour chaque Entités dans la liste sans savoir quelle vas être ce type car c’est l’appel à l’Entité qui vas ensuite déterminé vers quelle méthode de déplacement se dirigé. Ici le type de déplacement du Monstre est deplacerAleatoire donc il vas éxecuter la méthode seDeplacer( ) de la classe deplacerAleatoire.

Screen :



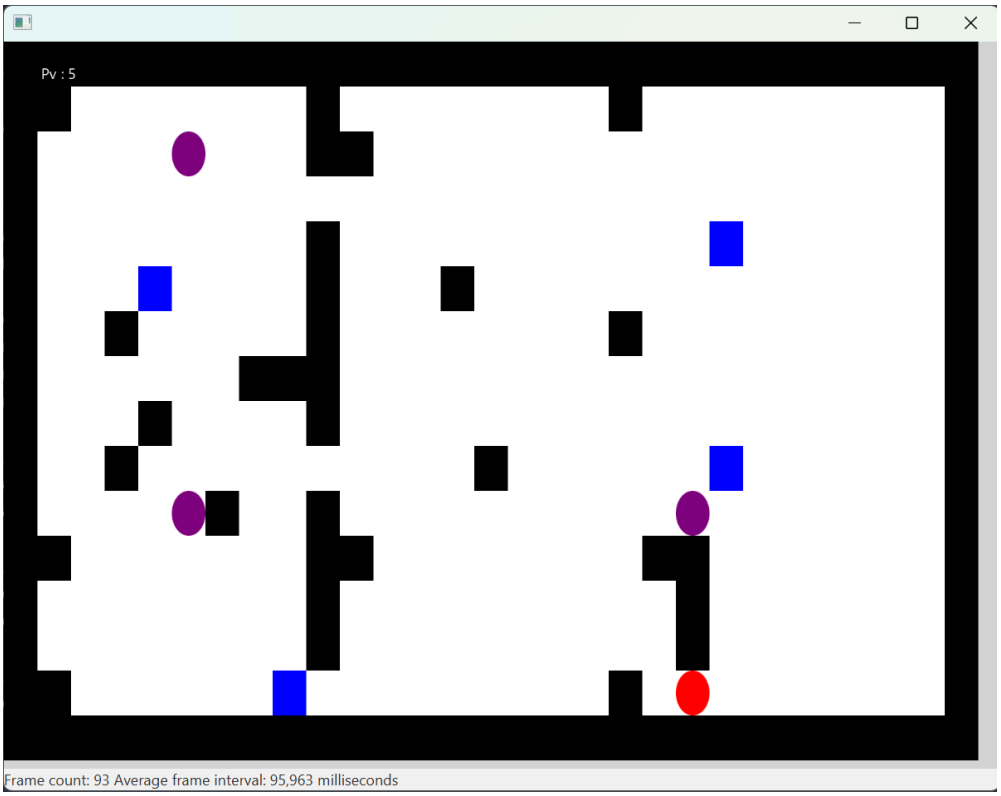
screen/screen1

Il y a trois Monstre avec des intelligence différente, c'est a dire qu'il utilise des méthode de déplacement différente. le monstre en haut à gauche à une intelligence nulle donc il reste immobile, celui en bas à gauche à une intelligence faible donc il bouge aléatoirement entre les 4 directions et enfin le monstre au centre à une intelligence moyens donc il se dirige vers le Perso sans prendre en compte les murs.



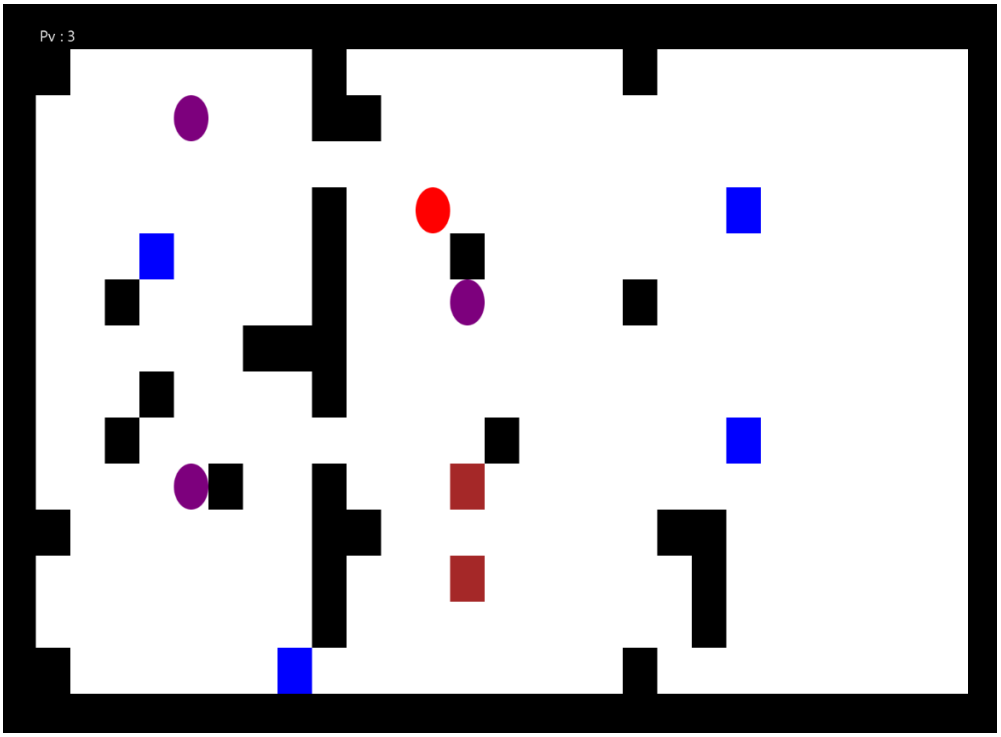
screen/screen2

Après quelque frames les monstres ont bougé et respect leurs intelligence, le monstre nulle reste immobile, le monstre faible à bouger sans prendre rien en compte complétement aléatoirement et le monstre moyens est allé vers le joueur.



screen/screen3

Si le joueur ce place derrière un mur, le monstre moyens reste bloquer sans bouger, ou plutôt en essayer de bouger dans un mur.



Mise en place des cases pièges (en brun) et des cases à effet (en blue)

Optimisation : Création de la classe `Element`

Elle va être extend par les classes `Entitées` et `Case`

Fonctionnalité 2.5 Monstres avec intelligence variable :

Classes à créer :

Interface `TypeDeplacement` possédant `seDeplacer(Entite, Laby)`

Une classe par type de déplacement (ex : `DeplacementManhattan`)

Classe Monstre——————————

Attributs à créer :

- Attribut `TypeDeplacement typeDeplacement`

<Interface> TypeDeplacement——————————

Méthodes à créer :

- seDeplacer(Monstre m,)

Méthodes à modifier :

Renommer `deplacerEntite` ⇒ `deplacerManuel` dans `Labyrinthe`

`DeplacerImmobile`
`DeplacerAleatoire`
`DeplacerManhattan`

Classe Labyrinthe——————————

Méthodes `boolean estTraversable(x,y)`

Critères de validation :
• Un monstre d'intelligence nulle reste immobile.
• Un monstre d'intelligence faible se déplace au hasard.
• Un monstre d'intelligence moyenne cherche à se rapprocher du héros sans prendre en compte les obstacles.
• Plusieurs monstres avec des intelligences différentes peuvent être présents dans le même labyrinthe.
• Chaque type de monstre (Orc, Fantome, Troll) peut avoir n'importe quel type d'intelligence. On peut avoir dans le même labyrinthe des Orc d'intelligence nulle, faible, moyenne et forte. Idem pour les trolls et les fantômes.

Tests à réaliser :

`Des monstres avec une intelligence différente sont présents en même temps`
`Les monstres d'intelligence moyenne construisent un chemin contenant les pièges`

`Les monstres d'intelligence élevée construisent un chemin sans pièges`

Quatrième itération :

Conception : Alexis, Jassem

Implémentation : Logan, Yanis

Séquence : Martin

Classe : Yanis, Alexis

Tests : Martin, Alexis, Logan

Fonctionnalité 3.2 Fantômes :

Classes à créer :

- `Fantome extends Monstre`

Classe Fantôme ——————————————————————

Attributs à créer :

- `x`
- `y`
- `TypeDeplacement`

Méthodes à créer :

- `peutTraverser(String dir)` : retourne `estTraversable()` de l'objet à la position suivante

Méthodes à modifier :

- SUPPRIMER `estTraversable(int x, int y)`

Critères de validation

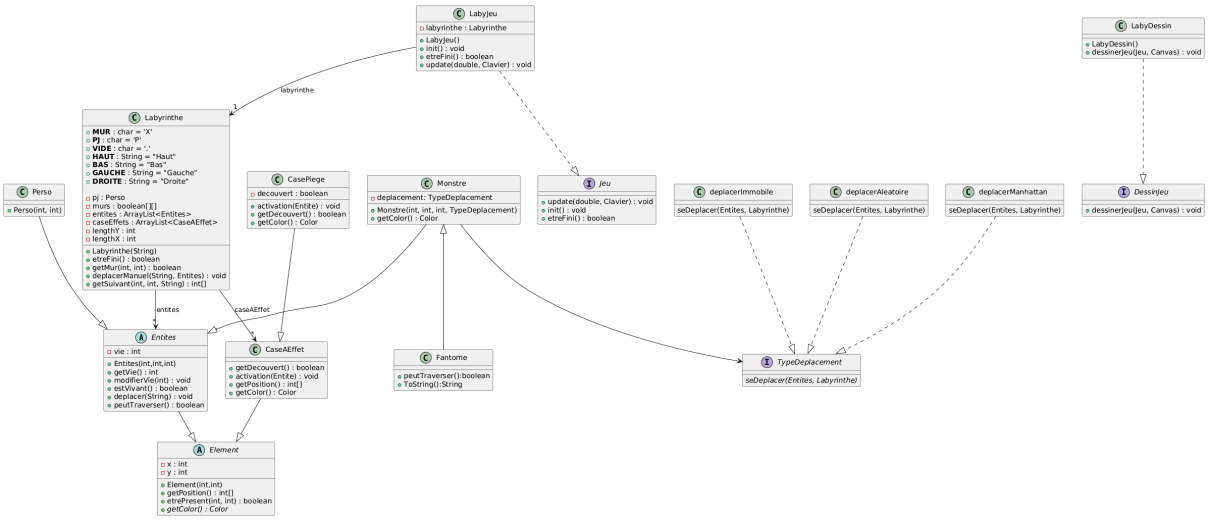
• Les fantômes peuvent traverser des murs mais pas les autres personnages (les autres monstres ou le héros)
• Les autres monstres continuent à être bloqués par des murs.
• Le Fantôme est représenté par 'F'

Tests à réaliser :

`Les fantomes traverse les murs`

`Les fantomes ne traverse pas les monstres`

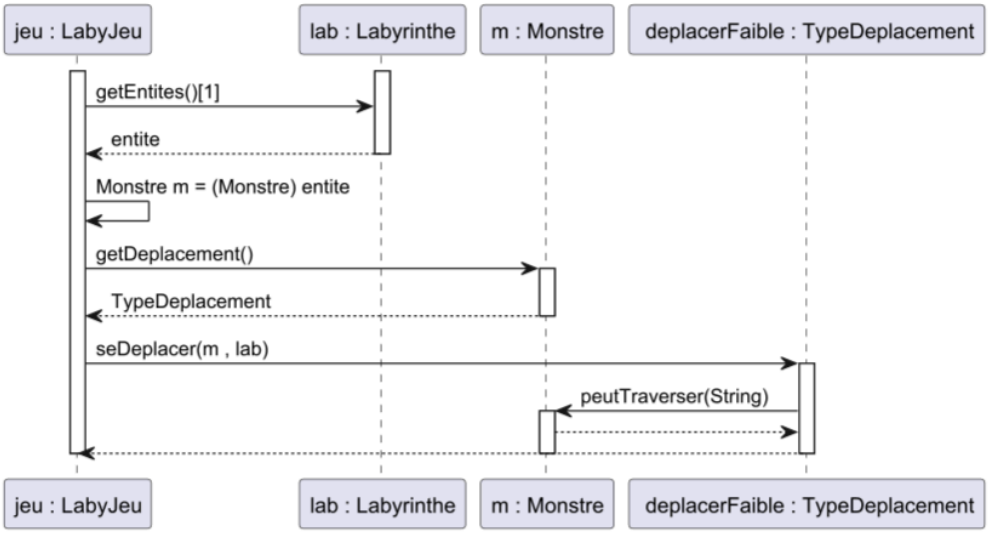
diagramme de classe :



CODE

Il a fallut modifier la méthode `estTraversable` initialement dans Labyrinthe, elle devient `peutTraverser(String, Labyrinthe)` dans `Entites` qui dit maintenant si l'entité correspondant peut traversé une case sur une direction avec maintenant une direction en `String` et un `Labyrinthe`

diagramme de séquence de `sedeplacer()` utilisant traversable :



Fonctionnalité 6.2 Mise en place de l'amulette :

Classe Labyrinthe————————--

Attributs à créer :

- `public static final char AMULETTE` = A

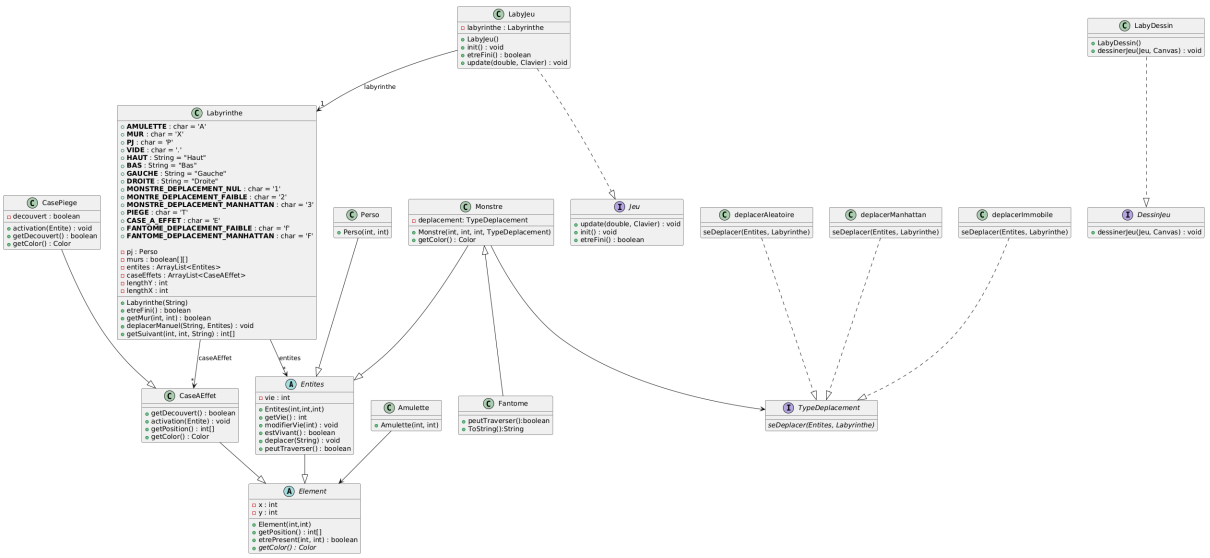
Classe `Amulette extends Elements` ————————

Attributs à créer :

- `x`

- `y`

diagramme de classe :



Critères de validation :
• L'amulette ne peut être placée que sur une case vide.
• Les monstres et le héros peuvent se situer sur la case de l'amulette.
• L'amulette est affichée dans le jeu sous la forme d'un cercle jaune sur la case vide où elle se trouve.

Tests à réaliser :

`L'amulette est placé correctement à l'initialisation`

`Les monstre et le héro peut traversé la case de l'amulette`

Fonctionnalité 6.3 Acquisition de l'amulette

Classe Entite————————-

Méthodes à modifié :

- `déplacer(String)` pour que l'amulette soit prit par le personnage

Classe Perso————————-

Attributs à créer :

- `possedeAmulette`

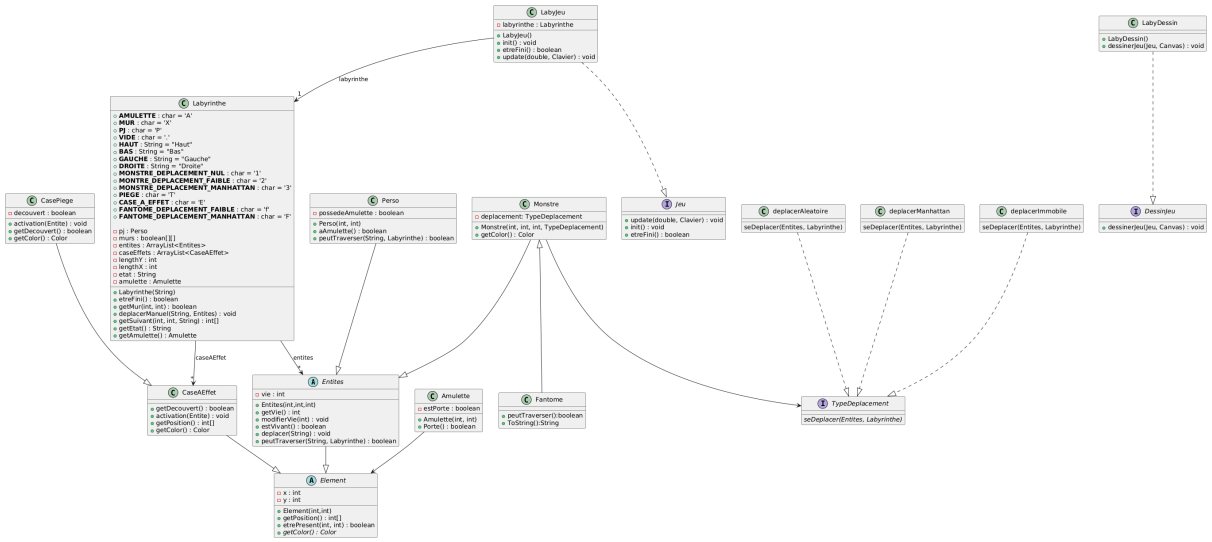
Méthodes à créer :

- `aAmulette()`

Classe `Labyrinthe` ————————————

- Ajouter la clé à la création du labyrinthe avec la lettre 'A'

diagramme de classe :



La méthode `deplacer()` de `Entite` dois être modifier pour que l'amulette soit prise seulement par le personnage. Il y a aussi l'ajout de l'attribut dans `Perso` `possedeAmulette` et le getter `aAmulette`

l'ajout d'un attribut `amulette` dans Labyrinthe pour éviter un `instanceof()` et d'un getter `GetAmulette()` qui retourne l'attribut amulette.

Critères de validation
- L'amulette n'est plus sur le plateau (et n'est plus affichée).
- Le Héros possède l'amulette.
- Un monstre ne récupère pas l'amulette

Tests à réaliser :

`L'amulette une fois prit, disparait du jeu`

`Le Personnage possède l'amulette`

`Les monstres ne peuvent pas récupérer l'amulette`

Cinquième itération :

Conception : Alexis, Jassem

Implémentation : Jassem, Martin

Séquence : Yanis

Classe : Alexis

Tests : Alexis, Logan

Fonctionnalité 6.4 Fin du jeu : victoire du héros

Classe Labyrinthe————————-

Méthodes à modifier :

`etreFini()` : si le perso est à sa position de base && `perso.possedeAmulette()`

Critères de validation :

Lorsque le héros possède l'amulette et qu'il retourne à l'entrée du labyrinthe, le jeu s'arrête et un message est affiché dans la console.

Lorsque le héros ne possède pas l'amulette, le jeu ne s'arrête pas même si le héros retourne à l'entrée.

Classe Labyrinthe————————————————-

Attributs :

- `String etat` : peut être “défaite”, “en cours”, “victoire”

Méthodes à créer :

- `victoire()`

Classe LabyJeu————————————————-

Méthodes à créer :

- victoire()

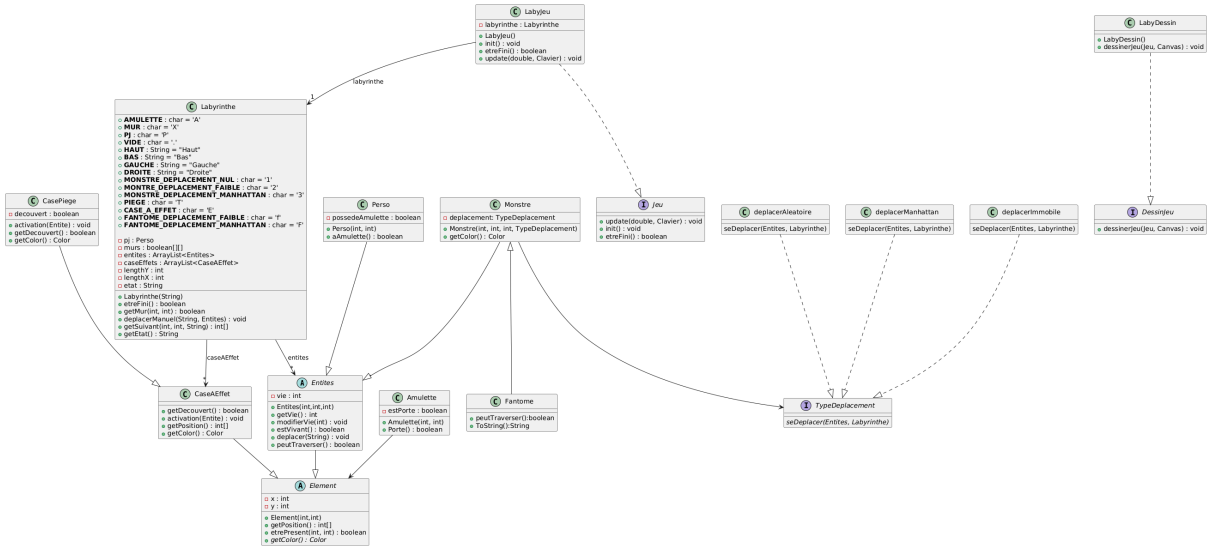
```
public void victoire() {
    laby.victoire();
  }
```

Class LabyDessin————————————-

Méthode à modifier :

- `ecranFin` écran différent selon ² `()`

diagramme de classe :



CODE

diagramme de séquence de `etreFini()` :

Tests à réaliser :

`Le jeu est terminé lors que le perso à ramener l’amulette`

`Le jeu continue si le perso passe sur l’entré sans l’amulette`

Présentation :

Dans cet ordre:

(Jassem) Factorisation avec Entite (Monstre et Perso)

(Alexis) Factorisation du code avec Element (Entite et CaseEffet)

(Yanis) Patron Strategie avec TypeDeplacement

(Martin) Changement methode peutTraverser de labyrinthe à entité (avec adaptation pour fantome)

(Logan) getColor pour faciliter l’affichage des Element